SigLIP · arXiv 2303.15343 · Sep 14

Archaeologist

seat 1 · Kanta · 11+3

Priors → SigLIP → PaliGemma → π_0 / OpenPI → BEHAVIOR

Paper club · VLA Architectures · dark academic

Sigmoid vs Softmax · the surgical swap

Spine of this paper: pairwise logistic replaces batch InfoNCE

CLIP Softmax (InfoNCE)

- Asymmetric i→t + t→i softmax
- Needs full-batch normalization
- All-gather / sync across devices
- Negatives compete exclusively
- Weak at small batch sizes
- Memory ~ $|B|^2$ logits + gather

SigLIP Pairwise Sigmoid

- Independent +1 / −1 per pair
- No global denom / no all-gather
- Chunked distributed loss (Fig.1)
- Learnable t + bias b (b≈−10)
- Strong at BS ≤ 1k–16k
- Saturates ~32k (larger can hurt)

Influential priors that shaped SigLIP

What this paper inherits — and surgically replaces

CLIP

Dual-encoder LIP setup SigLIP surgically swaps (softmax → sigmoid)

ALIGN

Large-scale noisy web image–text; scale narrative

LiT

Locked-image tuning path → SigLiT recipe

InfoNCE / SimCLR

Contrastive softmax lineage being replaced

ViT / SoViT

Vision backbone family; shape-opt → So400M

WebLI / PaLI

Data + multilingual / VLM context

Sigmoid-in-classif (ReaL)

Motivates pairwise logistic over exclusive softmax

FLIP / BASIC / CoCa / BLIP

Contemporaneous competing LIP recipes

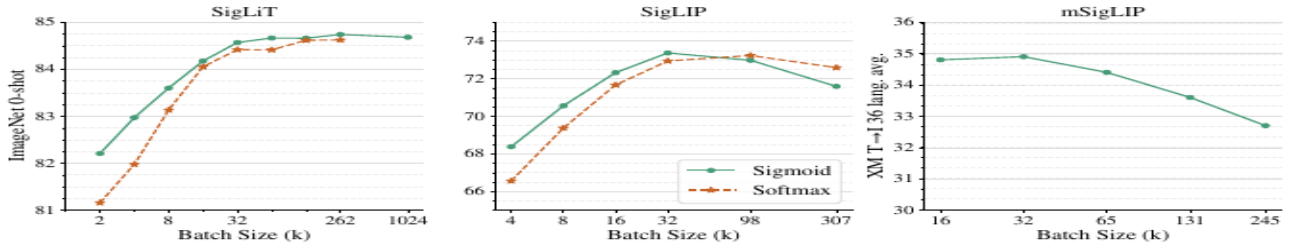


Figure 2: The effect of pre-training batch size. **Left: SigLiT results**, trained for 18B seen examples. Sigmoid loss outperforms the softmax loss significantly with small batch sizes, and performs similarly at larger batch sizes. We successfully trained an SigLiT model with up to *one million* batch size. However, performance for both sigmoid and softmax saturate at around 32 k batch size. **Middle: SigLIP results**, trained for 9B seen examples. Both sigmoid loss and softmax loss saturate at a reasonable batch size, while the peak of the sigmoid loss comes earlier and slightly outperforms the peak of the softmax loss. A very large batch size hurts both losses. **Right: mSigLIP results**, trained for 30B seen examples. With a multilingual setup using over 100 languages, 32 k batch size is surprisingly sufficient and scaling beyond that hurts performance on a 36-language cross-modal retrieval task.

method. In words, we first compute the component of the loss corresponding to the positive pairs, and $b - 1$ negative pairs. We then permute representations across devices, so each device takes negatives from its neighbouring device (next iteration of sum **B**). The loss is then calculated with respect to this chunk (sum **C**). This is done independently in each device, such that each device computes the loss with respect to its local batch b. Losses can then simply be summed across all devices (sum **A**). Individual collective permutes (for sum **B**) are fast (and indeed D collective permutes is typically faster than two all-gathers between D devices), and the memory cost at any given moment is reduced from $|\mathcal{B}|^2$ to b^2 (for sum **C**). Usually b is constant as scaling $|\mathcal{B}|$ is achieved by increasing the number of accelerators. Due to being quadratic with respect to the batch size, the vanilla loss computation rapidly bottlenecks scaling up. This chunked approach enabled training with batch sizes over 1 million on relatively few devices.

4. Results

In this section, we evaluate the proposed SigLiT and SigLIP models across a wide range of batch sizes. We discuss what can be achieved with a small number of accelerator chips, using both SigLiT and SigLIP recipes. We also briefly discuss the impact of batch size on multilingual language image pre-training. We ablate the importance of our large-batch stabilization modification and the introduced learned bias term and present a study on the effect of positive and negative pairs ratio in the sigmoid loss. Lastly, we explore SigLIP's data noise robustness.

To validate our models, we report zero-shot transfer results on the ImageNet dataset [14] and zero-shot retrieval results across 36 languages on the XM3600 dataset [44]. We use the ScalingViT-Adafactor optimizer [58] by default for all our experiments.

4.1. SigLiT: Scaling batch size to the limit

Following [59], we use the same precomputed embeddings for the images using a ViT-g vision model, and train a base size text tower from scratch with the same hyperparameters using the LiT image-text dataset [59].

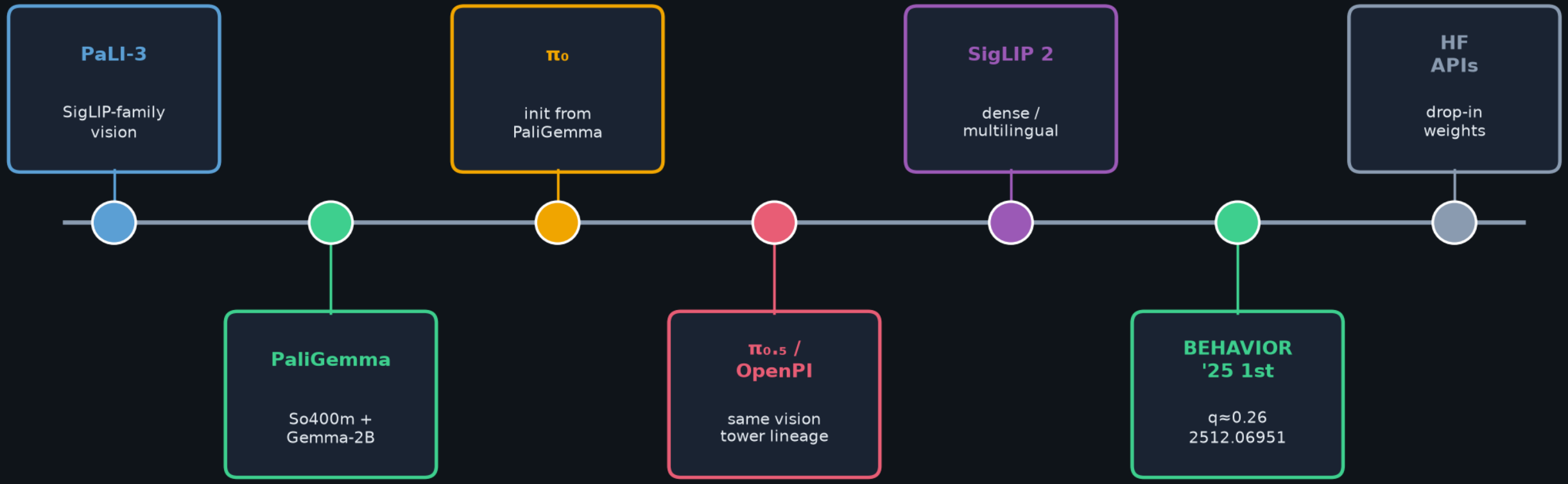
We perform a study over a wide range of batch sizes, from 512 to 1 M, demonstrating the impact of batch size for contrastive learning. Results are presented in Figure 2 (left). When the batch size is smaller than 16 k, sigmoid loss outperforms softmax loss by a large margin. With growing batch sizes, we observe that softmax loss quickly catches up and potentially slightly underperforms sigmoid loss with a large enough batch size. Overall, we recommend using the SigLIP recipe for large batch sizes as well, due to the simplicity, compute savings, and straightforward memory efficient implementation.

There is a consensus that contrastive learning benefits from large batch sizes, while most of the existing studies stop at 64 k batch size [59, 35, 10]. We successfully trained an SigLiT model at one million batch size, to explore the limit of contrastive learning. To our surprise, the performance saturates at 32 k batch size, further scaling up the batch size only gives a minor boost, and the model peaks at

Paper Fig. 2 — batch-size effect (sigmoid vs softmax)

Subsequent work that builds on SigLIP

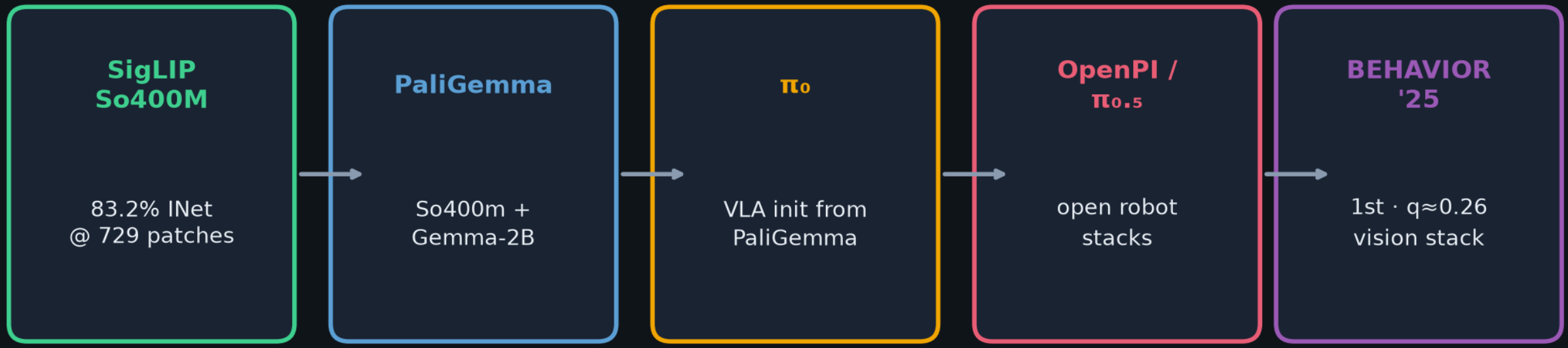
Clear line from vision tower → open VLA stacks → BEHAVIOR winners



Critical bridge: PaliGemma = SigLIP-So400m + Gemma → robot VLAs inherit the eyes

So400M → open VLA eyes

Public efficiency point that became the default vision tower



Perception backbone — NOT a classical motion planner

Chunking / stages / recovery / data still sit downstream of these eyes

One-sentence lineage

CLIP / ALIGN / LiT + WebLI
→ SigLIP → PaliGemma → π_0 / OpenPI
→ $\pi_{0.5}$ → 2025 BEHAVIOR winners
(and → SigLIP 2)

Honest hedges

Freeze/unlock of SigLIP weights in every OpenPI fork is not always documented.
“Uses SigLIP” ≠ isolating the sigmoid loss vs data / architecture / scale.

Club thread · long-horizon control

SigLIP IS

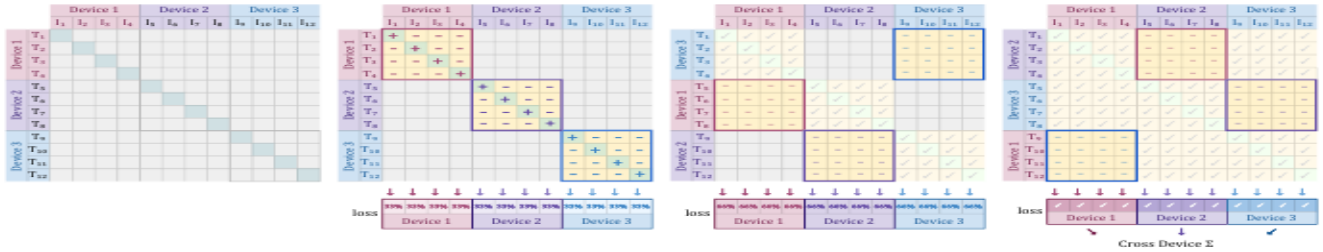
- Perception / vision tower
- Default open eyes for VLAs
- Standardized backbone
- Image–text pretrain recipe

SigLIP is NOT

- A motion planner / PDDL
- A substitute for chunking
- A recovery / System-2 loop
- Proof that sigmoid loss = VLA win

BEHAVIOR failures more often sit in chunking / stages / recovery / data

than in ImageNet-0 of the vision tower — but So400m is still the default open eyes those controllers see through.



(a) Initially each device holds 4 image and 4 text representations. Each device needs to see the representations from other devices to calculate the full loss.

(b) They each compute the component of the loss (highlighted) for their representations, which includes the positives.

(c) Texts are swapped across the devices, so device 1 now has $I_{1:4}$ and $T_{5:8}$ etc. The new loss is computed and accumulated with the previous.

(d) This repeats till every image & text pair have interacted, e.g. device 1 has the loss of $I_{1:4}$ and $T_{1:12}$. A final cross-device sum brings everything together.

Figure 1: **Efficient loss implementation** demonstrated via a mock setup with 3 devices and a global batch size of 12. There are no all-gathers, and at any point in time only the bright yellow square (size 4×4) is materialized in memory.

minimize the following objective:

$$-\frac{1}{2|\mathcal{B}|}\sum_{i=1}^{|\mathcal{B}|}\left(\overbrace{\log\frac{e^{t\mathbf{x}_i\cdot\mathbf{y}_i}}{\sum_{j=1}^{|\mathcal{B}|}e^{t\mathbf{x}_i\cdot\mathbf{y}_j}}}^{\text{image}\to\text{text softmax}}+\overbrace{\log\frac{e^{t\mathbf{x}_i\cdot\mathbf{y}_i}}{\sum_{j=1}^{|\mathcal{B}|}e^{t\mathbf{x}_j\cdot\mathbf{y}_i}}}^{\text{text}\to\text{image softmax}}\right)$$

where $\mathbf{x}_i = \frac{f(I_i)}{\|f(I_i)\|_2}$ and $\mathbf{y}_i = \frac{g(T_i)}{\|g(T_i)\|_2}$. In this paper, we adopt the vision transformer architecture [17] for images and the transformer architecture [47] for texts. Note that due to the asymmetry of the softmax loss, the normalization is independently performed two times: across images and across texts [36]. The scalar t is parametrized as $\exp(t')$, where t' is a global freely learnable parameter.

3.2. Sigmoid loss for language image pre-training

Instead of the softmax-based contrastive loss, we propose a simpler alternative that does not require computing global normalization factors. The sigmoid-based loss processes every image-text pair independently, effectively turning the learning problem into the standard binary classification on the dataset of all pair combinations, with a positive labels for the matching pairs (I_i, T_i) and negative labels for all other pairs $(I_i, T_{j\neq i})$. It is defined as follows:

$$-\frac{1}{|\mathcal{B}|}\sum_{i=1}^{|\mathcal{B}|}\sum_{j=1}^{|\mathcal{B}|}\underbrace{\log\frac{1}{1+e^{z_{ij}(-t\mathbf{x}_i\cdot\mathbf{y}_j+b)}}}_{\mathcal{L}_{ij}}$$

where z_{ij} is the label for a given image and text input, which equals 1 if they are paired and -1 otherwise. At initialization, the heavy imbalance coming from the many negatives dominates the loss, leading to large initial optimization steps attempting to correct this bias. To alleviate this, we introduce an additional learnable bias term b similar to the temperature t. We initialize t' and b to $\log 10$ and -10 respectively. This makes sure the training starts roughly close to the prior and does not require massive over-correction. Algorithm 1 presents a pseudocode implementation of the proposed sigmoid loss for language image pre-training.

3.3. Efficient "chunked" implementation

Contrastive training typically utilizes data parallelism. Computing the loss when data is split across D devices necessitates gathering all embeddings [59] with expensive all-gathers and, more importantly, the materialization of a memory-intensive $|\mathcal{B}| \times |\mathcal{B}|$ matrix of pairwise similarities.

The sigmoid loss, however, is particularly amenable to a memory efficient, fast, and numerically stable implementation that ameliorates both these issues. Denoting the per-device batch size as $b = \frac{|\mathcal{B}|}{D}$, the loss is reformulated as:

$$-\frac{1}{|\mathcal{B}|}\underbrace{\sum_{d_i=1}^{D}}_{\text{A: }\forall\text{ device }d_i}\overbrace{\sum_{d_j=1}^{D}}^{\text{B: swap negs across devices}}\overbrace{\underbrace{\sum_{i=bd_i}^{b(d_i+1)}}_{\text{all local positives}}\underbrace{\sum_{j=bd_j}^{b(d_j+1)}}_{\text{negs from next device}}\mathcal{L}_{ij}}^{\text{C: per device loss}}$$

This is particularly simple for the sigmoid loss as each pair is an independent term in the loss. Figure 1 illustrates this

Paper Fig. 1 / Algorithm 1 — chunked sigmoid loss

Method · three recipes

SigLiT

Locked pretrained ViT
Trainable text
LiT image–text data
Up to BS=1M
84.5% INet (g/14)

SigLIP

From-scratch ViT+text
WebLI English
Fair CLIP baseline
BS sweet spot 32k
So400M → 83.2%

mSigLIP

B/16 multilingual
Full WebLI 100+ langs
Same ~32k sufficiency
XM3600 retrieval
Vocab memory tricks

Archaeologist close

Priors → SigLIP → PaliGemma → π_0/OpenPI → BEHAVIOR · hedges intact